Moodle's Architecture

Class 02 Team 3

TABLE OF CONTENTS

01
Introduction

Introduction

- Learning Management System (LMS) designed to support teaching and collaboration.
- Created by Martin Dougiamas in his PhD project, in 1999.
- Existing systems (WebCT) were expensive, closed source — Martin wanted a system that was open for everyone to tinker with and improve

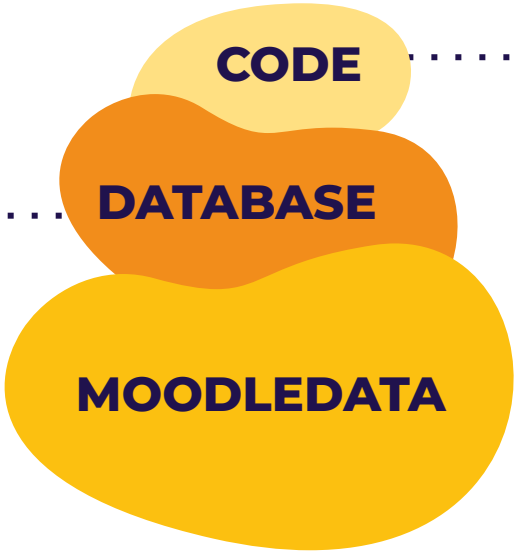
Introduction

400 MILLION+ Users

160,000+ registered sites across 240 countries

02
How Moodle works

CORE COMPONENTS



PHP

Scripts executed by the web server

RDBMS

Exemples: MySQL and PostgreSQL

Secure folder

Located outside the web root

CORE LAYERS

Presentatio n

Managed by themes and global UI objects that dictate the layout and appearance.

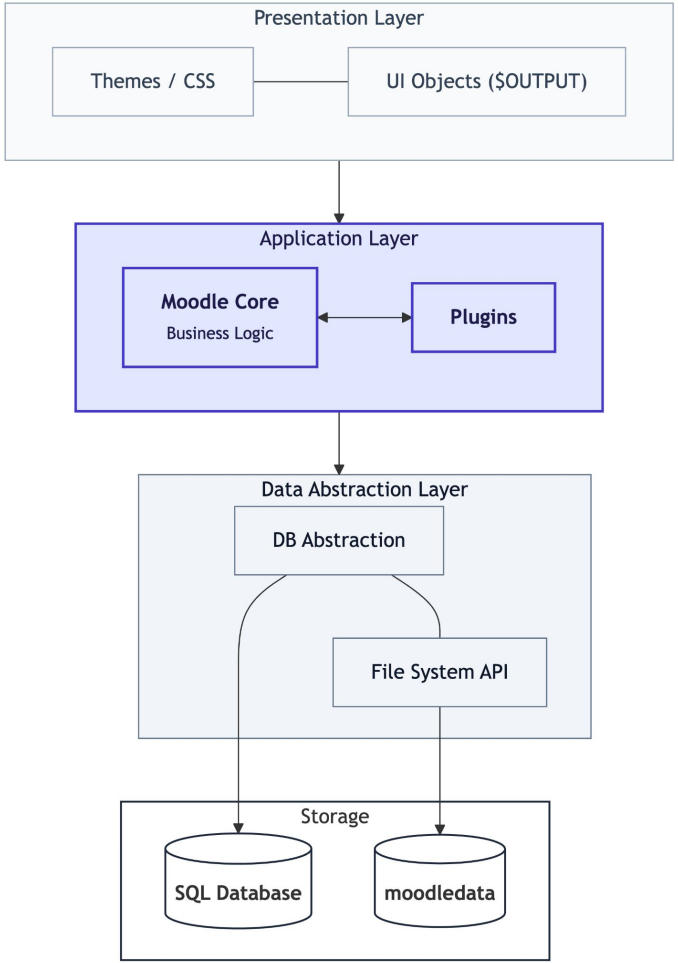
Application

“Fat core” logic and the extensive plugin ecosystem.

Data

Handled by the database abstraction and the moodledata file system.

CORE LAYERS


Presentation Layer
Themes / CSS
UI Objects ($OUTPUT)
Application Layer
Moodle Core
Business Logic
Plugins
Data Abstraction Layer
DB Abstraction
File System API
Storage
SQL Database
moodledata

Frankenstyle

To organize its massive number of plugins, Moodle uses a naming convention called Frankenstyle, which combines the plugin type and its name.

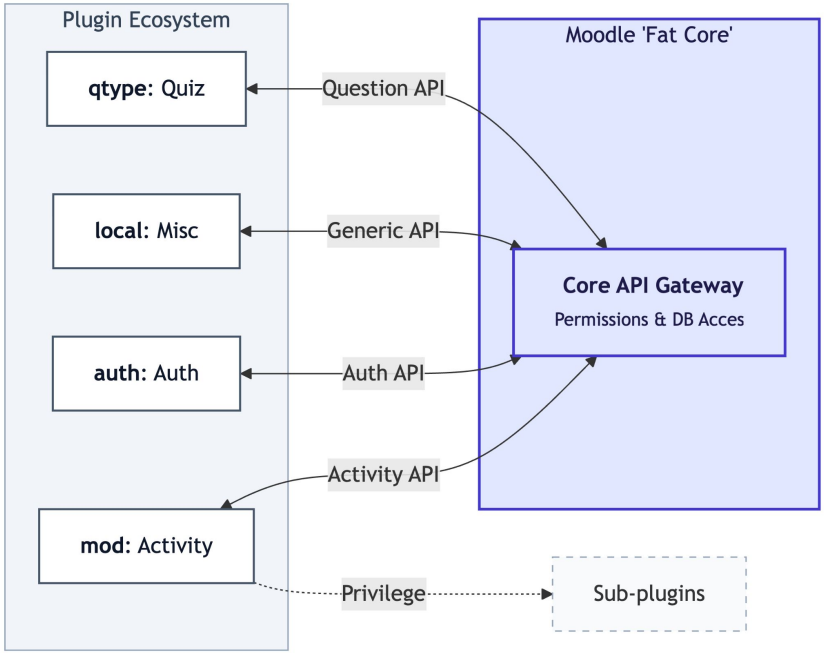
Plugin type	Plugin name	Frankenstyle	Folder
mod (Activity module)	`forum`	`mod_forum`	`mod/forum`
mod (Activity module)	`quiz`	`mod_quiz`	`mod/quiz`
block (Side-block)	`navigation`	`block_navigation`	`blocks/navigation`
qtype (Question type)	`shortanswer`	`qtype_shortanswer`	`question/type/shortanswer`
quiz (Quiz report)	`statistics`	`quiz_statistics`	`mod/quiz/report/statistics`

03
Architectural Patterns

Duality between Plugin-Based Pattern and Monolithic

Moodle is both plugin-based and monolithic. Its core can be extended with plugins to add features, but the core and all plugins run together and are deployed as a single system.

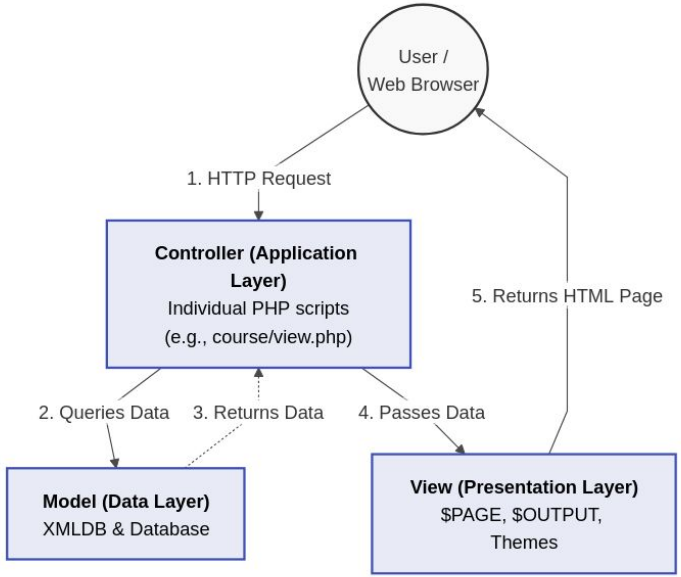
In this sense, Moodle is a **modular monolith**: modular extensions within one application.



MVC Pattern (Model-View-Controller)

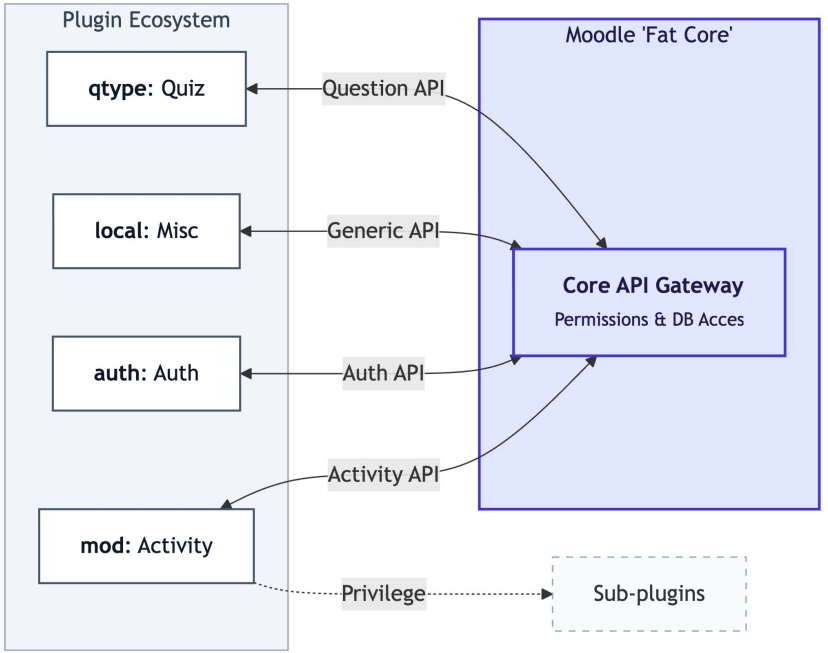
The MVC pattern in Moodle is implemented using the **Page Controller** approach suited to older PHP. Individual scripts (e.g., index.php, view.php) act as controllers handling requests and business logic.

The **Data Access/XMLDB layer** serves as the model for database operations, while the view is handled by global objects like $PAGE (page context) and $OUTPUT (generating the final HTML).



Duality between Plugin-Based Pattern and Monolithic

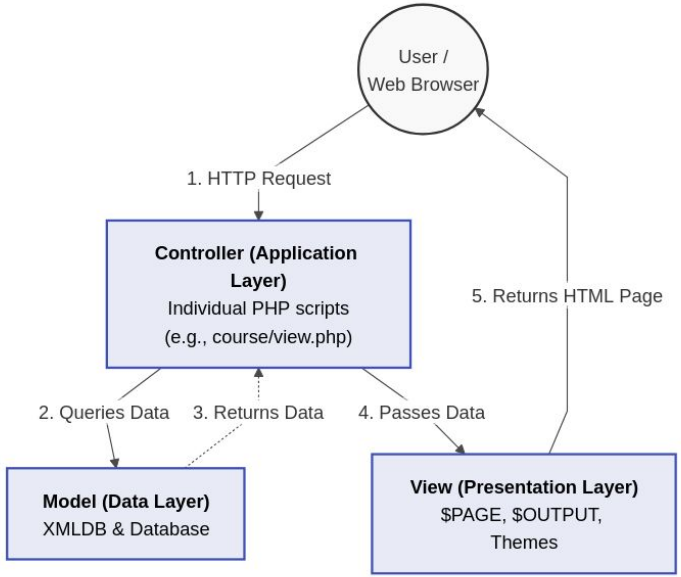
- Moodle's most prominent characteristic is its **Plugin-Based Architecture**;
- Resembles a **Microkernel Pattern**, but adapts it by featuring a **'fat core'**, rather than a minimal one;
- Vast majority of Moodle's actual educational features are entirely **self-contained plugins**;
- 'Fat Core' and Plugin Ecosystem run together and are deployed as a single unified unit, effectively creating a **Modular Monolith**.



MVC Pattern (Model-View-Controller)

Since Moodle was built to fit an older PHP style, it specifically implements MVC through the **Page Controller Pattern**:

- Instead of routing everything through a single index file (modern approach), the HTTP request goes directly to a specific script (e.g., index.php, view.php) and that script acts as the **page controller**;
- The **Data Access/XMLDB layer** serves as the **model** and handles the database interactions;
- Finally, the **view** is abstracted through global registry objects: $PAGE (contextual information) and $OUTPUT (generating the final HTML).



Registry Pattern

- The Registry Pattern is a way to store shared objects in a global place, so any part of the system can access them without passing them around
- Instead of passing objects through functions, Moodle creates global variables during startup (bootstrap phase via config.php)
- $CFG for configuration, $USER for the current session and $DB for the database connection are some of the several global objects instantiated

04
Quality Attributes

Quality Attributes

1

Modifiability

Easy to customize using plugins without changing the main code

2

Maintainability

Updates to the core system are safe and won't break your plugins

3

Portability

Works on many different types of computers and databases

4

Security

Fine-grained control over permissions for every user and activity

05
Conclusion

Conclusion

About Us / News /

Community | Moodle LMS | Product

Field Notes: Where AI meets learning in Moodle LMS

DECEMBER 16, 2025 BY LAUREN GOODMAN

At Moodle, we've always believed that great learning design starts with people. Technology should enhance learning and make it more engaging, memorable and effective. This is the idea behind our approach to AI. We build technology tools to serve human goals, not replace them.

Across the Moodle community, we're seeing educators and organisations explore AI in ways that fit their own context, whether that is saving time, sparking ideas, or making everyday teaching and learning a little smoother. But we also know that not everyone wants, or can rely on, AI tools. And we're building for them, too.

Thank You

And keep using Moodle